

stem_agent: A Differentiation Mechanism, Not a Universal Agent

github.com/TUe-Ray/stem_agent

1. What I chose to build

The stem cell analogy in the prompt does three things and I took each of them as a software constraint. The cell reads signals from its environment, which becomes the scenario. It runs an internal program of differentiation, which becomes the mutation loop. And it stays inside hard safeguards that pull it back when things go wrong, which becomes the Guardian kernel. The product of the project is not an agent that does any particular task. It is a mechanism that takes a scenario and grows a specialized harness for it, frozen at the point where adding more structure stops paying off. For a different class of tasks you start a new run.

To make that framing more than rhetorical, I built scenarios for five task classes: `toy_structured_answer`, `gsm8k`, `security_review`, `research_synthesis`, and `release_qa_triage`. The same loop has to produce structurally different harnesses across them. That is the kind of claim that only becomes interesting once it is measured.

2. What is mutable, and what is not

The architecture lives in the split between Nucleus and Guardian. Nucleus is mutable: it proposes one mutation to the genome per generation, and the genome encodes everything that defines what the agent is — roles, workflow steps, tool specs, generated tools, memory schema, quality gates, self-evaluation rubric, retry policy, stop rule, environment, artifacts. Guardian is immutable. It owns mutation validation, the sandbox for generated tools, fitness evaluation, hidden eval logging, budget limits, rollback and provenance, and signal-policy enforcement. None of those files (`guardian.py`, `evaluator.py`, `sandbox.py`, `versioning.py`, `budget.py`, `validators.py`, `signal_policy.py`) can be touched by the Nucleus. Self-evaluation inside the Nucleus is free to evolve; Guardian fitness is not.

End to end the loop runs:

```
scenario signals → Nucleus diagnosis → genome mutation → harness materialization → Guardian  
evaluation → GenomeArchive selection → either back to mutation or freeze the best verified harness.
```

Two choices behind that loop matter beyond what the diagram shows. The first is the rejection of greedy hill-climbing. Greedy gets stuck on the first local maximum; during an early prototype I watched it throw out a promising harness branch because one recent mutation had a slightly worse score than its parent. The replacement is a Top-K GenomeArchive that keeps several strong, diverse genomes alive in parallel, so new mutations can branch off whichever ancestor fits the current metric deficit best. This is closer in spirit to MAP-Elites than to plain evolutionary search.

The second is the stop rule. The interesting thing about a stem cell is that it eventually commits. When evolution finishes, the system freezes `frozen_genome.yaml`, and that artifact is what executes future tasks. Evolution is not the product; the specialized harness is.

3. Information policy

A loop that evolves toward a score will overfit to whatever signal it can see. So the boundary between what the Nucleus is allowed to see and what it is not had to be made explicit rather than implicit. Three layers:

- Layer 0, always visible to the Nucleus: scenario task description, current genome structure, generation number, mutation type, and direction of the last evaluation (`improved`, `degraded`, or `neutral`).
- Layer 1, configurable, on by default: aggregate score, previous aggregate score, score delta.
- Layer 2, never visible to the Nucleus: validation case inputs, expected outputs, per-case scores, Guardian criterion breakdowns, hidden eval scores, hidden case details.

The framing in the README is the one I keep returning to: Nucleus can see the dashboard, not the answer key.

This is enforced in `src/stem_agent/kernel/signal_policy.py` and surfaced both as scenario-level configuration and as a CLI override. A `compare-ablations` command runs the same scenario with Layer 1 enabled and disabled and writes a comparison report. The ablation tooling matters because the policy is only meaningful if it is falsifiable; without it, "no signal leak" is just an intention.

Getting this right was harder than describing it. Several Nucleus prompt fields casually contained per-case feedback during early iterations. The policy module is the place where I forced myself to enumerate every channel the Nucleus could read from and put each one behind a gate.

4. Scenarios

A scenario folder has six possible files and every split has a role:

```
scenario.yaml
train_cases.jsonl
validation_cases.jsonl
hidden_cases.jsonl          # optional, observability only
external_benchmark_cases.jsonl # optional, auxiliary promotion signal
final_holdout_cases.jsonl    # optional, only after freeze
```

Train drives mutation. Validation drives promotion. Hidden cases are logged but never shown to the Nucleus and never contribute to promotion score; they exist to measure leakage without enabling it. External benchmark is an auxiliary signal mixed into promotion at a configured weight. Final holdout runs exactly once, after freeze, and appears only in the report.

The five scenarios serve different purposes. `toy_structured_answer` came first and is where the loop's own design got debugged (Section 5). `gsm8k_demo` and `gsm8k_full` are grounded math reasoning with verifiable answers; `init-benchmark gsm8k_full` downloads the official GSM8K data and writes fixed-seed splits of 100/100/200. The demo variant is a smoke test, not a benchmark report. `security_review_demo`, `research_synthesis_demo`, and `release_qa_triage_demo` cover three different task classes — threat-model triage, evidence-and-uncertainty synthesis, release go/no-go with rollback planning — and exist to test whether the same loop produces structurally different specialized harnesses across them. The Makefile target `evolve-robustness-demo` runs the three back to back, because the cross-domain claim is only interesting if there is something to compare.

5. How the loop actually learns

The first end-to-end run did not improve. Baseline and final scores landed in the same place. The mutation system was running and mutations were being applied, but nothing was being kept. That run is the most useful thing that happened in the project, because the failure had three distinguishable causes and all three were in the signal, not in the agent.

First, the evaluator was measuring the wrong thing. It rewarded markdown structure — a summary section, a steps section, a final-answer section. A bland but well-formatted reply already maxed out most metrics at baseline. There was no headroom for "more useful" to register.

Second, `min_delta = 0.03` was larger than the granularity of real improvements. Most actual gains showed up at the third or fourth decimal place. The threshold was rounding away every honest improvement.

Third, the failure analyzer only fired on explicit failures. On a plateau where nothing was broken but nothing was improving either, it had nothing to say. With no failure signal to anchor on, the mutation planner defaulted to retry-policy tweaks and whole-genome replacements; neither addresses "answers are too generic."

The fix was structural and propagated to later scenarios. The evaluator became scenario-sensitive, with twelve metrics covering qualities a structured answer can actually get better at: requirement coverage, format validity, constraint adherence, actionability, input specificity, self-review usage, quality-gate usage, generated-tool usage, plus cost and complexity penalties. Weights were rebalanced so that "well-formatted but vague" no longer wins. The failure analyzer was extended to read low-scoring metrics as well as explicit failures; a low `self_review_usage` proposes a review step, a low `input_specificity` proposes a revise step, a high

`complexity_penalty` blocks role and tool additions. `min_delta` came down to 0.005 for this scenario, and CLI scores went from four decimal places to six because a reporting bug had been masquerading as a learning failure. A few small kernel stability fixes followed: an epsilon in `should_promote` so improvements exactly equal to `min_delta` weren't rejected by floating-point comparison, and a circular import in `kernel/___init__.py`.

6. Results

Scenario	Baseline	Final (internal)	Holdout	Promoted
toy_structured_answer (openai_002)	0.1049	0.9727	0.9221	6
security_review (security_001)	0.9505	0.9521	0.9512	0
research_synthesis (research_001)	0.8239	0.7531 †	0.9223	0

† research_001 final score is a re-evaluation of the gen-0 genome at gen-1; see text.

toy_structured_answer is where differentiation happened. OpenAI run `openai_002` promoted six mutations in order: `modify_self_evaluation`, `modify_environment`, `add_workflow_step review_against_requirements`, `add_workflow_step revise_final_output`, `add_quality_gate required_sections_gate`, and `create_tool requirement_sections_checker`. The Guardian rolled back an unsafe generated tool, caught by the sandbox import-policy check, and a redundant role. The harness grew an internal review / revise / gate loop and a domain-specific checker tool, and the Guardian rejected the mutations that would have weakened it. 85 of 85 tests pass.

security_review baseline was already 0.950 on a minimal two-step genome. The reason is in the scenario's evaluation criteria: ninety percent of the weight sits on `section_check`, so any output that contains the required sections captures nearly the full score on its own. With only ten percent left on heuristic quality, `min_delta` = 0.015 is unreachable. Every mutation was rolled back or rejected; the Guardian correctly refused two whole-genome replacements that stripped required criteria. The frozen harness is the original minimal genome — the structurally right outcome reached for the wrong reason. The evaluator ceiling prevented any gradient from forming in the first place.

research_synthesis surfaces a different failure mode. No genome changed across the run, yet the validation score swung from 0.916 to 0.688 between generation 0 and generation 1 on identical harness structure. That is a 0.228 spread from LLM response variance alone, larger than any meaningful `min_delta`. The holdout score of 0.922, evaluated once on the frozen gen-0 genome against fresh cases, confirms the genome itself is stable; the apparent regression is noise. The run was force-frozen after repeated API hangs on the external benchmark call. Both robustness scenarios point at the same root cause as the original toy diagnosis: `section_check`-dominated evaluators that either ceiling out at baseline or get drowned by LLM evaluation variance. The diagnosis generalizes; the fix has not been propagated yet.

7. What surprised me

The agent was rarely the bottleneck. Almost every change that produced learning was in the connective tissue: scoring, reporting, the path from "output produced" to "harness changed." The mutation system was capable of doing useful work the whole time. It just had nothing to climb.

CLI display precision mattered more than I would have guessed. When real improvements live at the fifth decimal, rounding to four collapses them into "no progress." Trust in the loop dropped during the first run mostly because the report was telling a different story than the underlying numbers.

Adding more metrics initially made things worse before they got better. Twelve metrics with un-tuned weights produced a noisier signal than three crude ones. The fix wasn't more metrics; it was the right weights on the right metrics, with deliberate down-weighting of the easy ones so headroom existed for the harder ones to matter.

Signal policy was easier to state than to enforce. "Nucleus sees the dashboard, not the answer key" is one sentence. Actually getting there meant enumerating every channel the Nucleus could read from and putting each one behind a gate. Early prompt drafts leaked per-case feedback in places I hadn't considered. Without the explicit enumeration the boundary is just a vibe.

8. What failed or got cut

An LLM-judge metric for every internal evaluation. Too expensive, too noisy, and the variance washed out small promotions. It is now opt-in and only used for the external benchmark and final holdout.

Letting the Nucleus see external benchmark prompts so it could "understand what to aim for." That is signal leak by another name. Dropped quickly.

Greedy hill-climbing as the search method. Replaced with the Top-K archive after watching one good harness branch get thrown out because a single recent mutation was slightly worse than its parent.

Offline deterministic mode as the default. The deterministic runner is fine for tests, but the differentiation that emerges with a real LLM is qualitatively different from what the stub produces. The default flipped to OpenAI required; deterministic stayed only as a test-only fallback.

9. What I would do with more time

The most direct next step is applying the scenario-sensitive evaluator fix to the robustness scenarios. The same change that unblocked `toy_structured_answer` — replacing `section_check`-dominated criteria with fine-grained quality metrics and rebalancing weights away from surface format — has to be propagated to `security_review` and `research_synthesis`. Before that comparison is meaningful, the noise floor of LLM-based evaluation needs to be measured directly: same genome, repeated evaluations, multiple seeds. Right now `min_delta` is a guess relative to a variance no one has measured.

A formal stopping criterion. Today evolution runs until a step budget; the stem-cell analogue is wrong on that. A cell stops differentiating when its environment stops asking for more, not when a clock runs out. A better stop combines holdout plateau, complexity penalty trending up, and promotion frequency trending down. The agent "knows it is good enough" when adding more structure no longer pays off on a set it cannot see.

A real diversity metric on the archive. Top-K by score is one step beyond greedy but still collapses similar genomes. A behavior characterization — number of roles, depth of workflow, whether self-review is enabled, whether generated tools are present — would let the archive function as a quality-diversity grid in the MAP-Elites sense.

A bigger ablation grid: signal policy $\times$ `min_delta` $\times$ archive size $\times$ scenario. The infrastructure is in place; the missing input is compute budget. And the structural question that the framework hasn't been pushed on yet: GSM8K is a useful grounded benchmark, but the more interesting test of "stem agent" is a task class where the harness has to grow tools the framework wasn't pre-fitted for — browsing, retrieval, citation checking, code execution. That is where the differentiation mechanism either earns the name or doesn't.